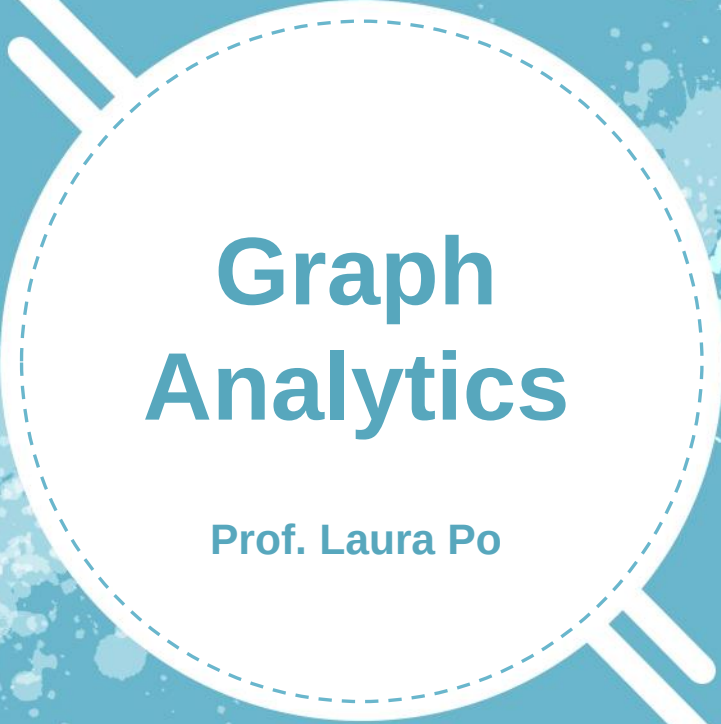




Graph Analytics

Prof. Laura Po

- Source code for the examples is taken from the book “Graph-Powered Machine Learning” written by Alessandro Negro.
- The code is available for download from the publisher’s website at <https://www.manning.com/books/graph-powered-machine-learning>, or you can clone it from my Git repository at <https://github.com/alenegro81/gpml>.



Collaborative Filtering

e-commerce site

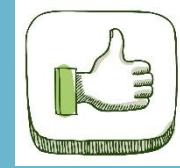
We want to design graph models and the algorithms on top to implement a recommender system for an e-commerce site that uses a collaborative filtering approach to gently suggest to users items that could be of interest to them.

The site sell many types of items, such as books, computers, watches, and clothing. The **details about each item are not curated**; some items have only a title, one or more pictures, and a small and useless description. On the site, **users** can buy items only if they are **registered and logged in**. Little information is tracked about each user—only what's necessary for payments and shipping.



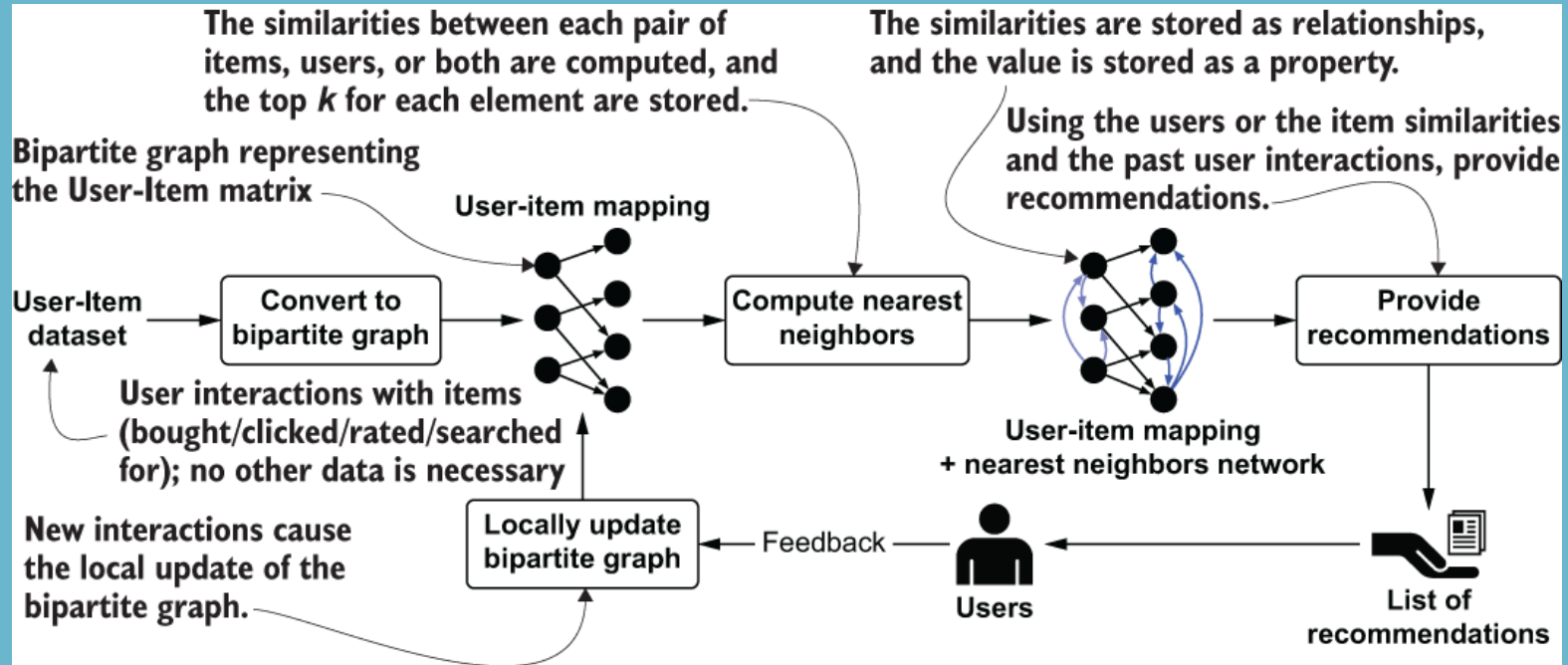


- no data useful for creating profiles about users or items is available.



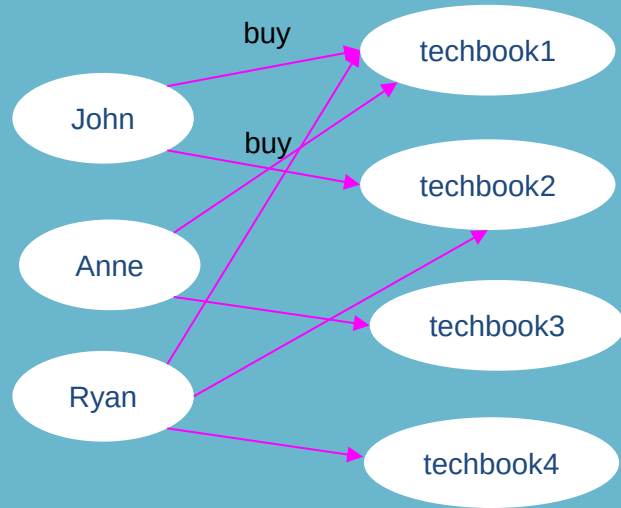
- users' activity is tracked: each user is logged in most of the time
- it is possible to collect and store a huge variety of interactions between users and items (purchases, clicks, searches, and ratings)

A graph-powered collaborative filtering recommender system

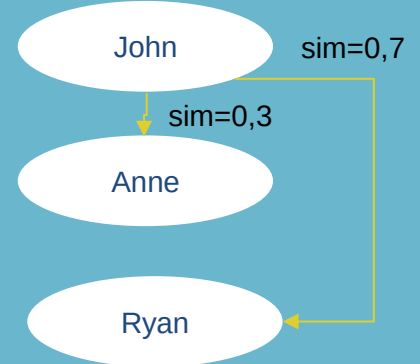
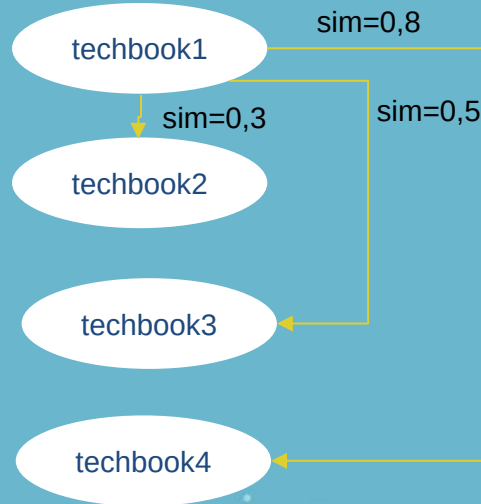


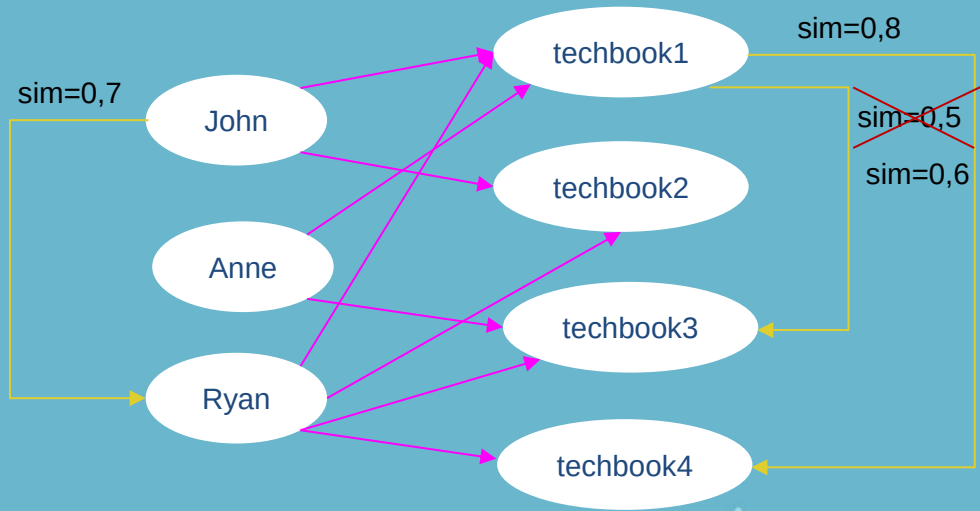
- Graphs are used for data representation for both the User-Item dataset and the nearest neighbor network

User- Item graph



Nearest neighbors networks

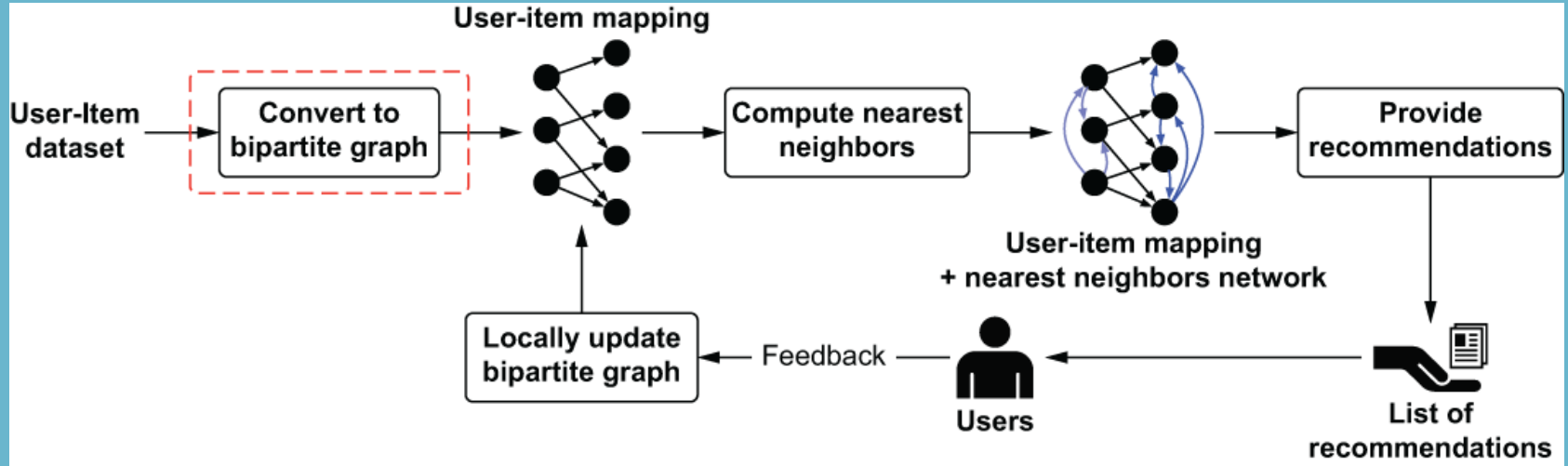




The image features a solid teal background. In the center, there is a white circle with a dashed white border. Inside this circle, the words "e-commerce" and "dataset" are written in a bold, white, sans-serif font, stacked vertically. Surrounding the central circle is a white, irregular, splatter-like shape. Several light blue diagonal lines are scattered across the teal background, some appearing as double lines.

e-commerce dataset

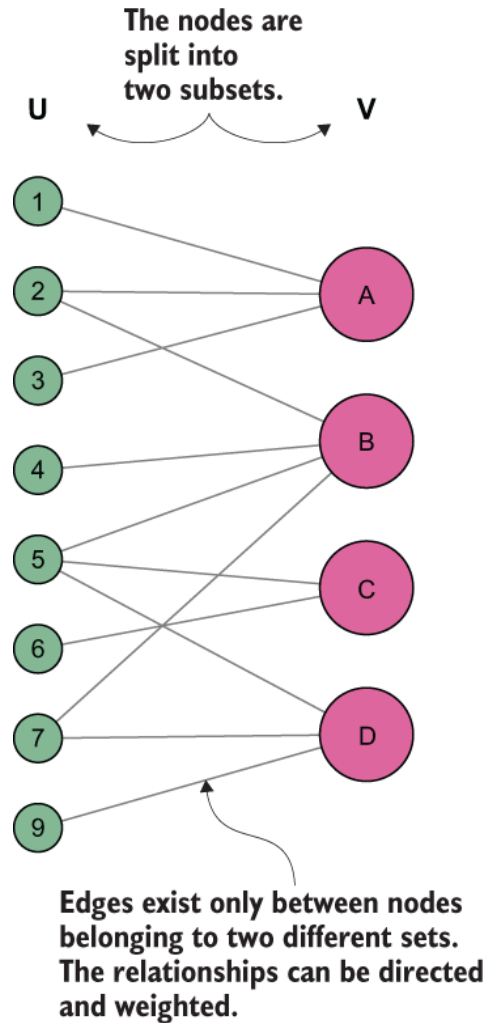
Bipartite graph creation in the recommendation process



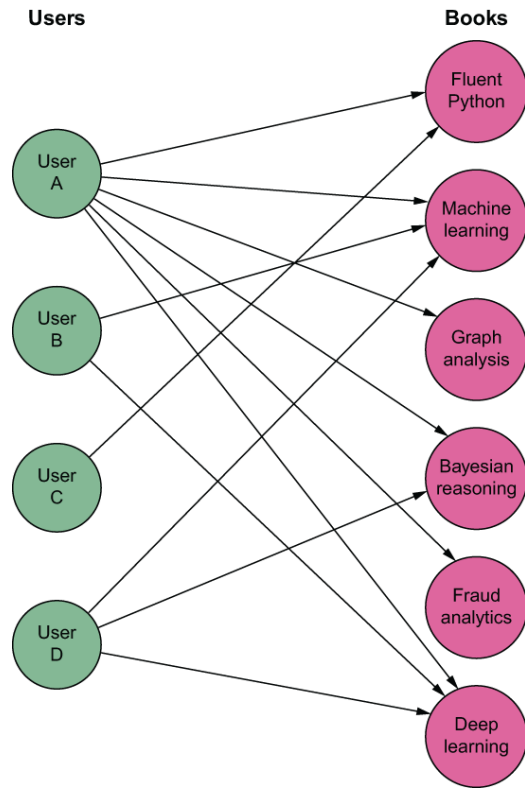
Example User-Item dataset for the e-commerce scenario

	Fluent Python	Machine Learning: A Probabilistic Perspective	Graph Analysis and Visualization	Bayesian Reasoning	Fraud Analytics	Deep Learning
User A	1	1	1	1	1	1
User B	0	1	0	0	0	1
User C	1	0	0	0	0	0
User D	0	1	0	1	0	1

Generic Bipartite Graph



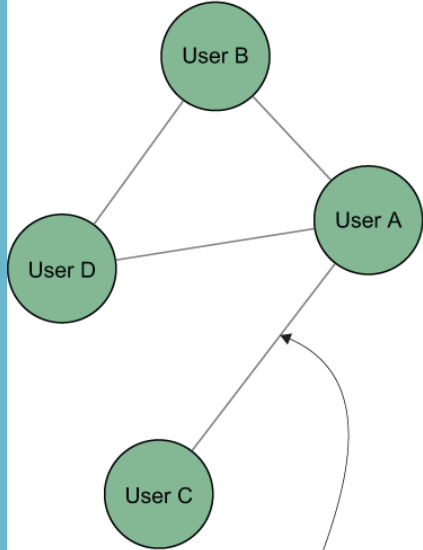
User-Item Bipartite Graph



it is possible to infer connections between nodes of the same type, creating a one-mode projection

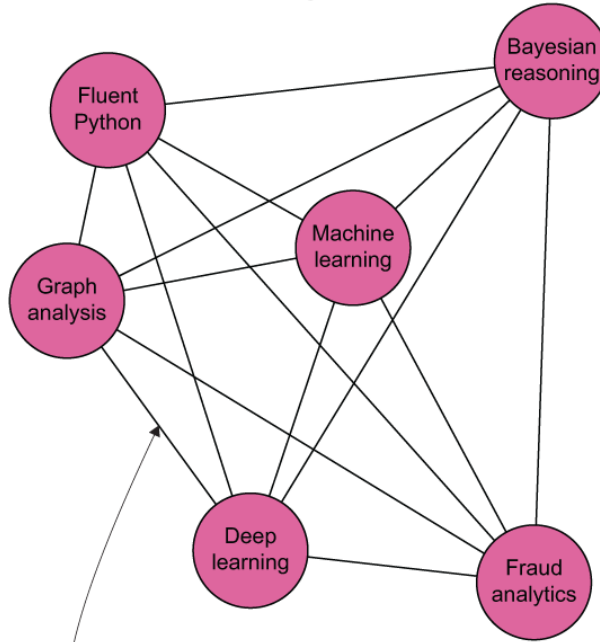
Projections

U - Projection



A relationship here exists if two nodes have a connection to the same node in V.

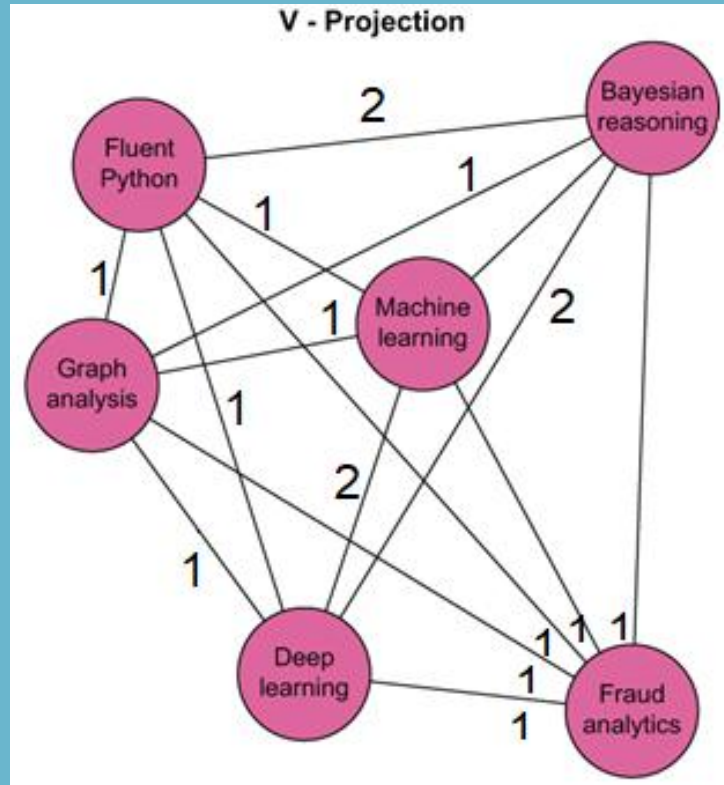
V - Projection



A relationship here exists if two nodes have a connection to the same node in U.

- What are the advantages? What is shown?
- What are the disadvantages? What is hidden?

A possible weighted projection of the graph



TO RECAP

WHAT ARE THE ADVANTAGES OF A BIPARTITE GRAPH REPRESENTATION OF A USER-ITEM DATASET?

Advantages

- It represents the data in a compact and intuitive way. The User-Item dataset is sparse by nature; generally, it has a lot of users and a lot of items, and users interact with a small portion of the items available. A matrix representing such interactions would contain a huge number of zeros and few useful values.
- Projections derived from the bipartite graph are information rich and allow different types of analyses both graphical (after visualization) and via algorithms (an example is graph clustering). E.g. customer segmentation and item clustering.
- The representation can be extended by modeling multiple bipartite graphs that have the same set of vertices but use different edges. This extension helps us represent data used as input to a multimodal recommendation engine. Engines use multiple types of interactions to provide better recommendations.
- When the nearest neighbor network is computed, it can be stored, sharing the user and item nodes of the bipartite graph. This approach simplifies the recommendation phase by providing access to a single data structure that contains both user preferences and the nearest neighbors network.

**the Retail
Rocket
dataset**

Creating a real dataset

Imports data from the Retail Rocket dataset (<https://retailrocket.net>), converting the User-Item matrix to a bipartite graph. The data is available through Kaggle at <http://mng.bz/8W8P>. If you have downloaded data from git () you can find it in the folder `\ch05\imports\retail_rocket`

1. Download and unzip the Retailrocket recommender system dataset (<https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset?source=download>) (305 MB)
2. Create a new Graph project in neo4j (example: GraphRetailRocket)
3. Start the project in Neo4J
4. In the folder where the python code is inserted, add the util folder and the config.ini file
5. Launch the import adding the path to the dataset with command

```
python import_retail_rocket_ui.py <Retailrocket_dataset_path>
```

In the end, you get this message: 2457 lines processed

- At the end of the import, which should take a few seconds, it is possible to run the following query and visualize a portion of the graph:

```
MATCH p=()-[r:PURCHASES]->() RETURN p LIMIT 25
```

- What can you see from the graph?
- Is the graph connected? Can you see multiple subgraphs?
- How many users are in the graphs?
- How many items are in the graphs?

Querying the model

With the newly created database, run queries to do the following:

- 1- What did userId 911093 buy? Who are the other users who made the same purchase?
- 2- Find the best sellers (the items with the highest number of purchases)
- 3- Find the best buyers (the users with the highest number of purchases)
- 4- Find recurrent buys (items purchased more than one time by the same user)
- 5- Who are the users who bought item 360200? What did these users buy in addition to this item?
- 6- Explore the multiple purchases of the user with userId: "890980" for item with itemId:"213834"

1- What did userId 911093 buy? Who are the other users who made the same purchase?

```
MATCH(u:User {userId:"911093"}) RETURN u
```

2- Find the best sellers (the items with the highest number of purchases)

```
MATCH (u:User)-[p:PURCHASES]->(i:Item) RETURN i.itemId, count(DISTINCT u) as num_User ORDER BY num_User DESC
```

3- Find the best buyers (the users with the highest number of purchases)

```
MATCH (u:User)-[p:PURCHASES]->(i:Item) RETURN u.userId, count(DISTINCT i) as num_Item ORDER BY num_Item DESC
```

4- Find recurrent buys (items purchased more than one time by the same user)

```
MATCH (u:User)-[p:PURCHASES]->(i:Item) WITH i.itemId as gitem, u.userId as guser, count  
(DISTINCT p) as num_occurrence RETURN gitem, guser, num_occurrence ORDER BY num_  
occurrence DESC
```

5- Who are the users who bought item 360200? What did these users buy in addition to this item?

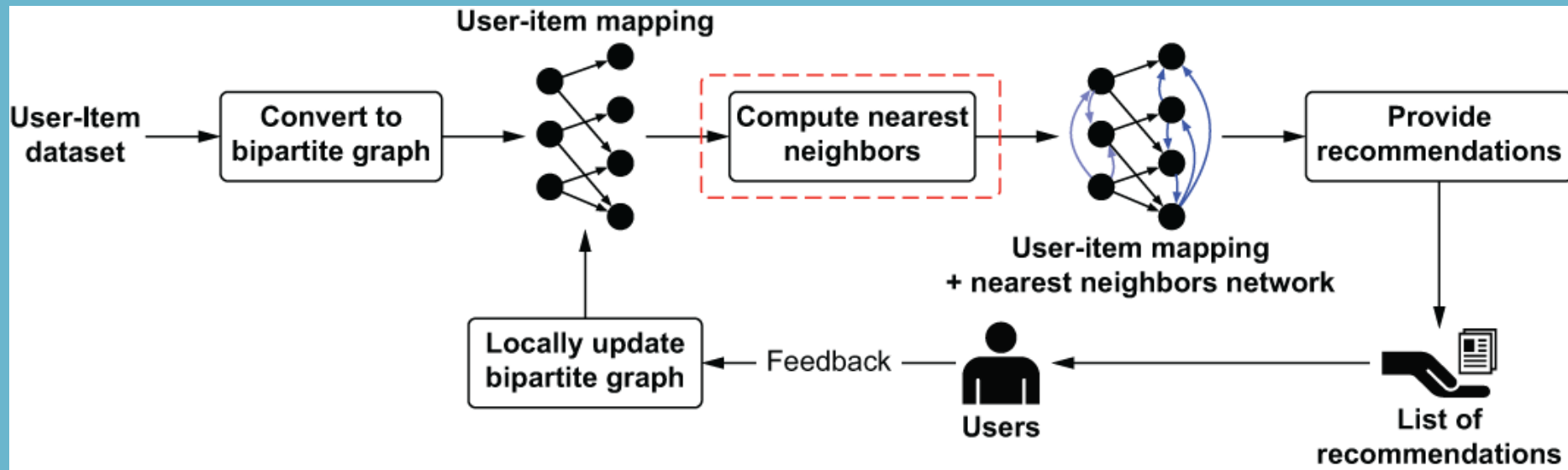
```
MATCH(u:User {userId:"360200"}) RETURN u
```

+ visual exploration

6- Explore the multiple purchases of the user with userId: "890980" for item with itemId:"213834"

```
MATCH g=(u:User {userId: "890980"})-[p:PURCHASES]->(i:Item {itemId:"213834"}) RETURN  
N g
```


**Computing
the nearest
neighbor
network**



2 approaches for computing the NNN

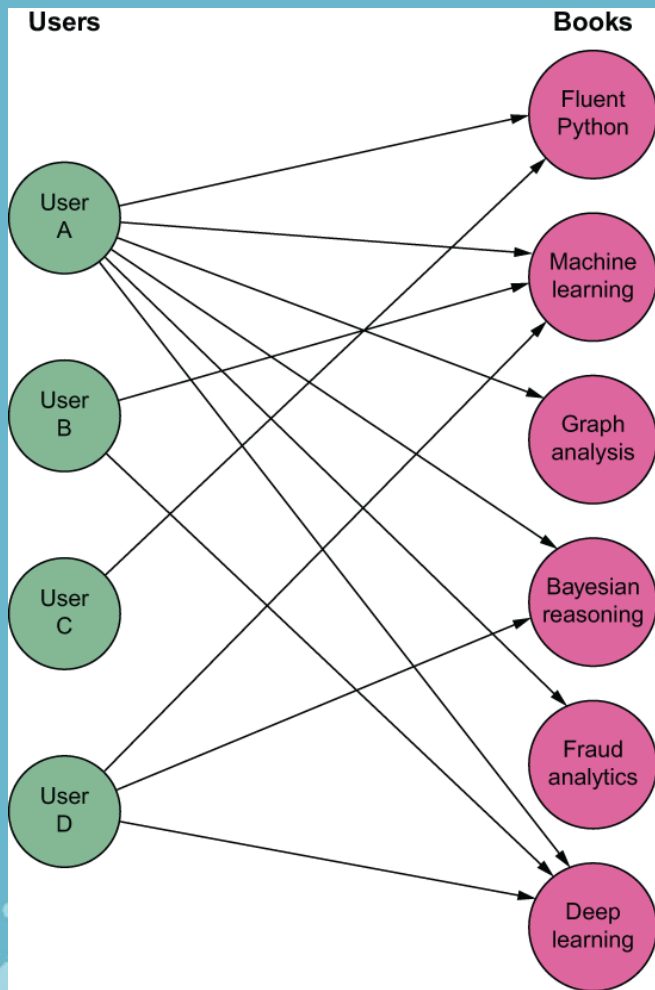
There are two possible approaches to memory-based recommendation for collaborative filtering:

- **Item-based**—The similarities are computed between items based on the users who interact with them (rating, buying, clicking, and so on).
- **User-based**—The similarities are computed between users based on the list of items they interact with.

Similarity computation

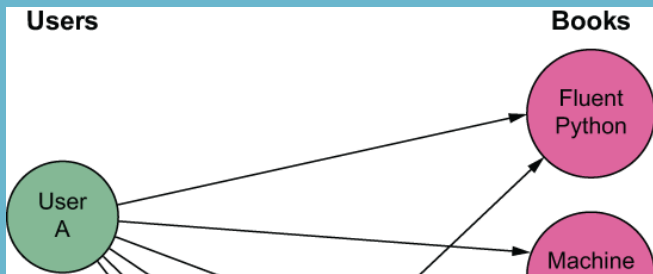
The procedure for the similarity computation is the same as that for the content-based approach:

1. Identify/select a similarity function that allows you to compute distances between homogeneous elements in the graph (among users or items, for example).
2. Represent each element in a way that's suitable for the selected similarity function (e.g. cosine similarity)
3. Compute similarities, and store them in the graph.



User vectors table

	Fluent Python	Machine Learning: A Probabilistic Perspective	Graph Analysis and Visualization	Bayesian Reasoning	Fraud Analytics	Deep Learning
User A	1	1	1	1	1	1
User B	0	1	0	0	0	1
User C	1	0	0	0	0	0
User D	0	1	0	1	0	1



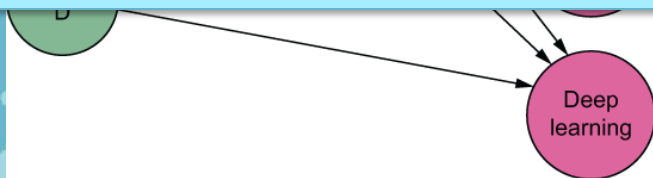
Compressed representation of the users in the VSM:

$Vector(User A) = [1, 1, 1, 1, 1, 1]$

$Vector(User B) = [0, 1, 0, 0, 0, 1]$

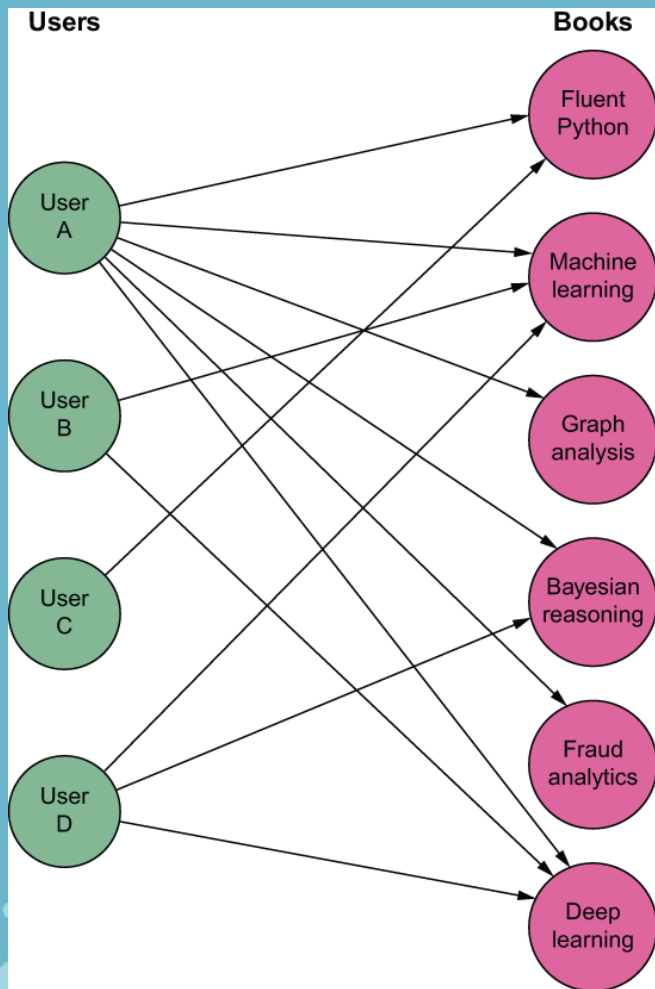
$Vector(User C) = [1, 0, 0, 0, 0, 0]$

$Vector(User D) = [0, 1, 0, 1, 0, 1]$



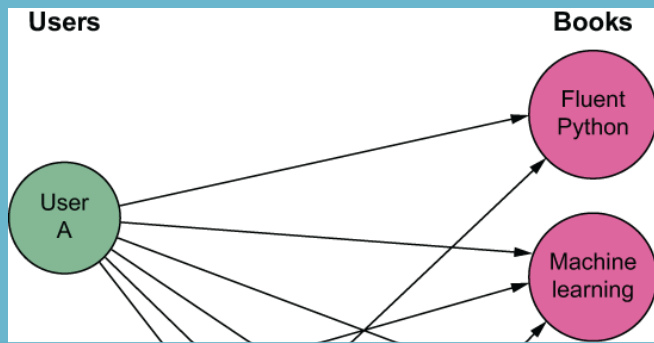
User vectors table

	Fluent Python	Machine Learning: A Probabilistic Perspective	Graph Analysis and Visualization	Bayesian Reasoning	Fraud Analytics	Deep Learning
User A	1	1	1	1	1	1
User B	0	1	0	0	0	1
User C	1	0	0	0	0	0
User D	0	1	0	1	0	1



Item vectors table

	User A	User B	User C	User D
Fluent Python	1	0	1	0
Machine Learning: A Probabilistic Perspective	1	1	0	1
Graph Analysis and Visualization	1	0	0	0
Bayesian Reasoning	1	0	0	1
Fraud Analytics	1	0	0	0
Deep Learning	1	1	0	1



Item vectors table

	User A	User B	User C	User D
Fluent Python	1	0	1	0
Machine Learning: A Probabilistic Perspective	1	1	0	1
Graph Analysis and Visualization	1	0	0	0
Bayesian Reasoning	1	0	0	1
Fraud Analytics	1	0	0	0
Deep Learning	1	1	0	1

$\text{Vector}(\text{Fluent Python}) = [1, 0, 1, 0]$

$\text{Vector}(\text{Machine Learning}) = [1, 1, 0, 1]$

$\text{Vector}(\text{Graph Analysis}) = [1, 0, 0, 0]$

$\text{Vector}(\text{Bayesian Reasoning}) = [1, 0, 0, 1]$

$\text{Vector}(\text{Fraud Analytics}) = [1, 0, 0, 0]$

$\text{Vector}(\text{Deep Learning}) = [1, 1, 0, 1]$



sparseness VS denseness

- Representing vectors in a memory-efficient way and accessing their values efficiently are important machine learning tasks.
- In the content-based scenario, vector creation for items relies on a small number of possible vector dimensions. The sparseness or denseness of a vector's data is the most important consideration.

A vector with relatively many nonzero values in relationship to its size is called *dense*.

Dense vector															
0.5	0.2	0.32	0.48	0.34	0.256	0.50	0	0.91	0.64	0.77	...	0.32	0.19	0.46	0.87
1	2	3	4	5	6	7	8	9	10	11	...	102	103	104	105

Sparse vector															
0	0	0	0	0.48	0	0	0	0	0	0.23	...	0	0	0.64	0
1	2	3	4	5	6	7	8	9	10	11	...	102	103	104	105

Vector representation

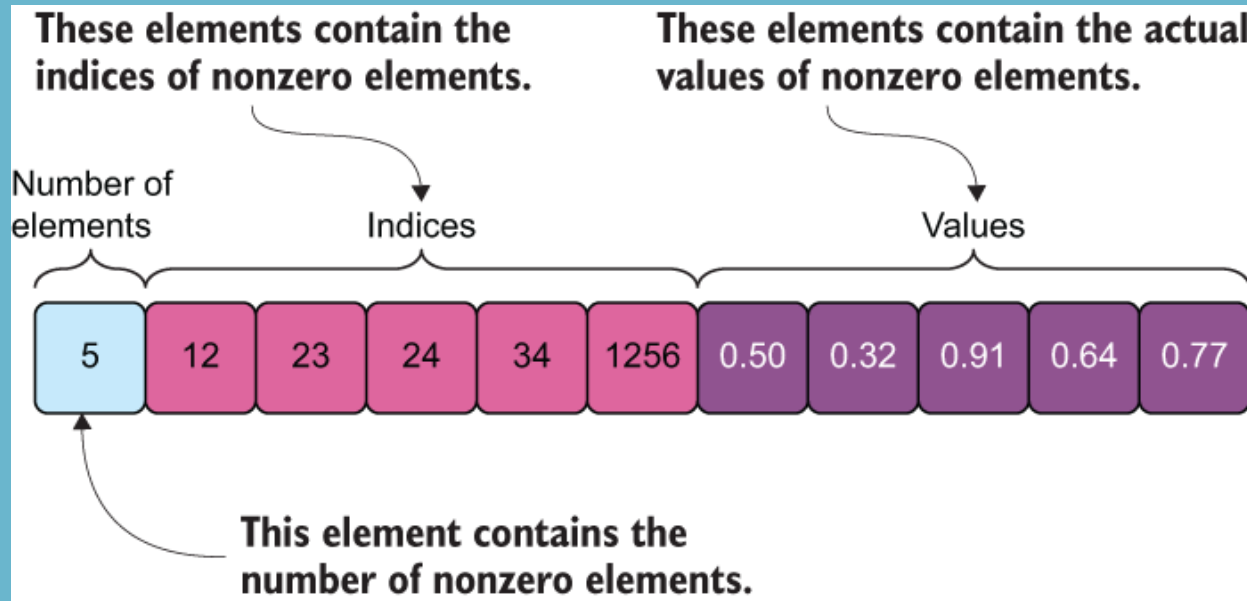
The best approach to vector representation varies according to the nature of the vector and how it will be used.

- A vector with relatively many nonzero values in relationship to its size is called **dense**.
- Such a vector can be represented by an implementation that stores values in an array of doubles or floats. Vector indices correspond directly to array indices. A dense vector's advantage is speed: it's quick to access and update any of its values.

Vector representation

- Vector, in which the total percentage of nonzero values is small, is called a **sparse** vector.
- Representing sparse vectors with an array is not only a waste of memory, but also makes any **computation expensive**.
- It is possible to represent a sparse vector in different ways to optimize memory and simplify manipulation. In Java, some of the most common methods are available in Apache Mahout (see, for example, <http://mng.bz/EVjJ> or <http://mng.bz/N8jD>), a distributed linear algebra framework used for creating scalable performant machine

Compact vector representation



ADV: with a single small array, it is possible to represent a long and complex sparse vector

See code
ch05/java directory
util/sparse_vector.py

**IS THERE A THRESHOLD
TO CONSIDER FOR USING
SPARSE VECTOR?**

Queries

- 1- Write a query for extracting the nonzero elements of each vector for users – try the query for {userId: "121688"}
- 2- Write a query for extracting the nonzero elements of each vector for items – try the query for {itemId: "37029"}

1- Write a query for extracting the nonzero elements of each vector for users – try the query for {userId: "121688"}

```
MATCH (u:User {userId: "121688"})-[:PURCHASES]->(i:Item)
```

```
return id(i) as index, 1 as value
```

```
order by index
```

--alternative for a weighted vector

```
MATCH (u:User {userId: "121688"})-[p:PURCHASES]->(i:Item)
```

```
return id(i) as index, count(DISTINCT p) as value
```

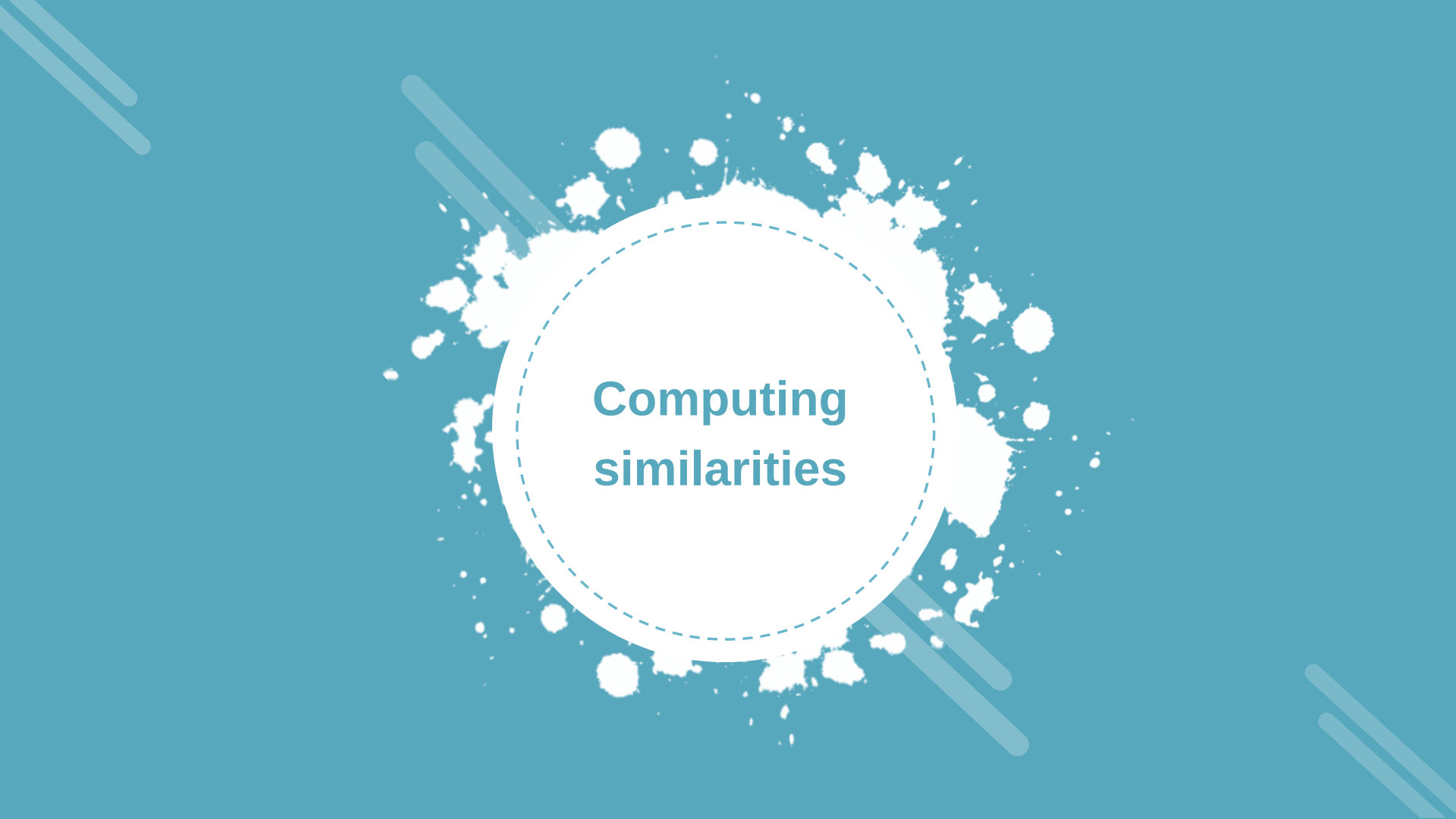
```
order by index
```

2- Write a query for extracting the nonzero elements of each vector for items – try the query for {itemId: "37029"}

```
MATCH (i:Item {itemId: "37029"})<-[:PURCHASES]-(u:User)
```

```
RETURN id(u) as index, 1 as value
```

```
ORDER BY index
```

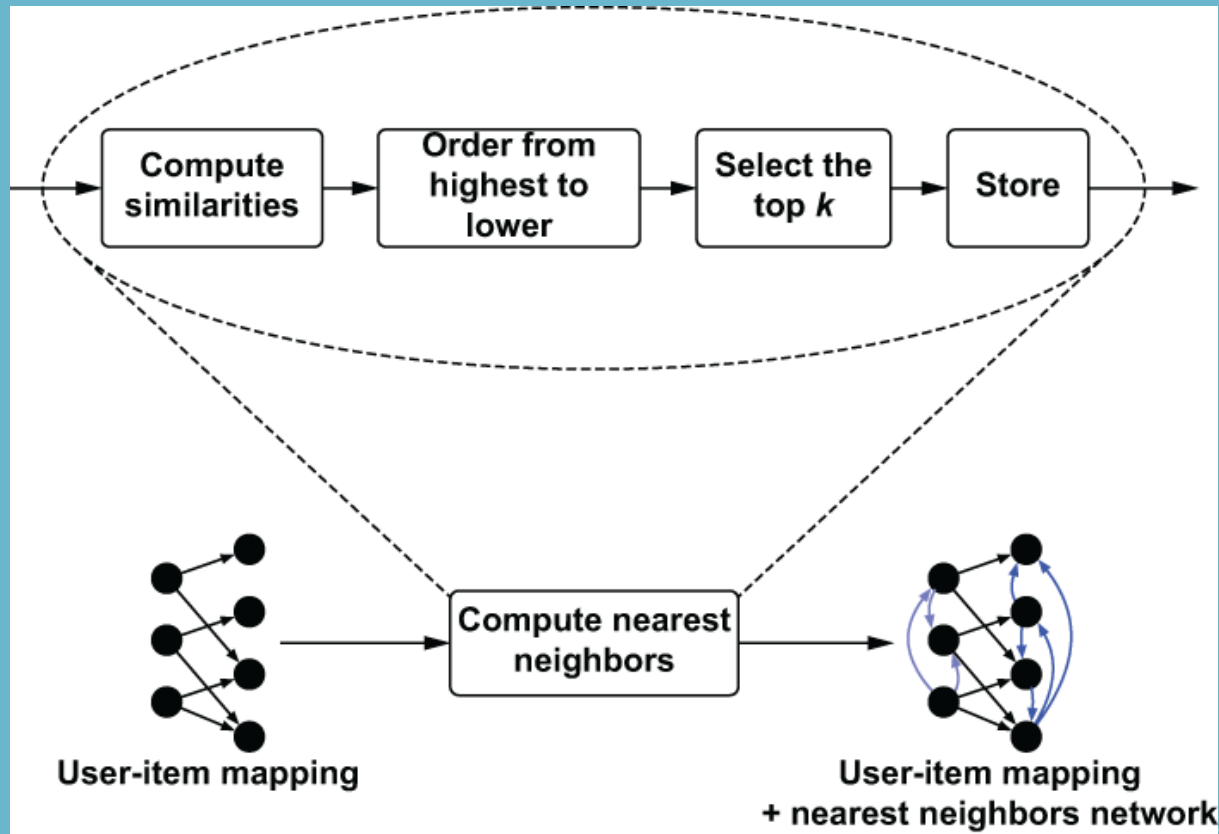


Computing similarities

Computing similarities

The next step is computing and storing the similarities. For each user or item, we have to do the following:

1. **Compute the similarities** with all the other elements (homogeneously: each user with other users and each item with other items).
2. **Order the similarities** in a descending order.
3. **Keep only the top k**, where the value of k is predefined. Alternatively, fix a threshold or minimum similarity value, and keep only the similarities above it.
4. **Store the top k** similar elements in the graph as new relationships between users or items.



The code is available in the `ch05/recommendation/collaborative_filtering/recommender.py` file.

**WHAT IS THE COST OF COMPUTING
SIMILARITIES?**

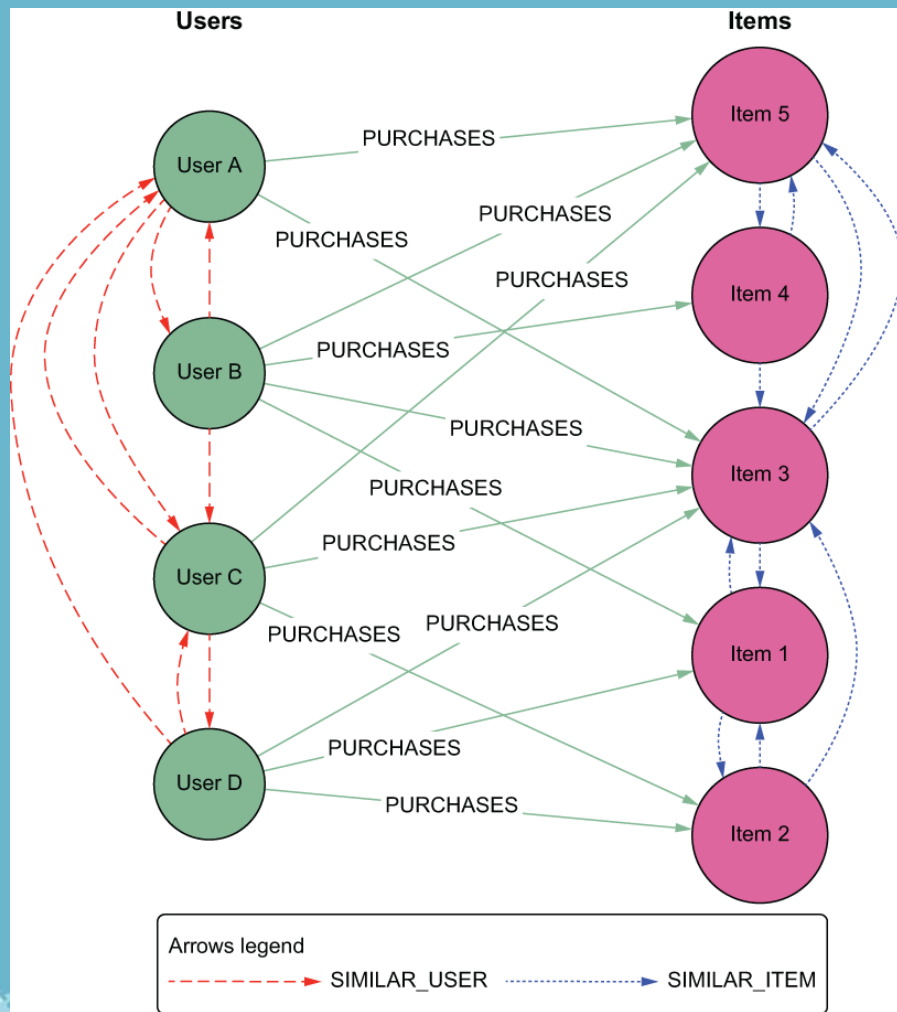
WHAT IS THE COST OF COMPUTING SIMILARITIES?

...given $|U|=|User| = 11719$, and $|V| = |Item|=12025$

140.920.975

ONE BIG GRAPH

The graph in figure is not a bipartite graph anymore, because there now exist relationships among elements in the same partition



3 SUB GRAPHS

The graph can be split into three highly relevant subgraphs:

- The subgraph with both U and I (U represents all the users, and I represents all the items) as nodes and only the PURCHASES relationships is the bipartite graph we had before.
- The subgraph with U as nodes and SIMILARITY as the relationship is the nearest neighbor network for U (the k-NN for U).
- The subgraph with I as nodes and SIMILARITY as the relationship is the nearest neighbor network for I (the k-NN for I).

These graphs are used as input for the last task, providing recommendations, but it is important to notice that these graphs already contain a lot of information.

New knowledge from the bipartite graph

The process distills new knowledge from the bipartite graph and stores it back in a way that can serve various purposes in addition to recommendations

- **Clustering items**—By applying some graph clustering algorithms on the items' nearest neighbor network, it is possible to recognize (for example) groups of products that are generally bought together or movies that are watched by the same group of users.
- **Segmenting users**—The same clustering algorithms can be applied on the users' nearest neighbor network, and the result will be a group (segment) of users who generally buy the same products or see the same movies. [User segmentation is the process of separating users into distinct groups, or segments, based on their shared characteristics.]
- **Finding similar products**—The items' nearest neighbor network itself is useful. If a user is looking at a specific product, by querying the graph it is possible to show a list of similar products based on the SIMILARITY relationship, and this operation will be fast.

TODO @HOME

Using the code provided, create new graphs that contains also the KNN.

Then, find the following:

1. Given an item, the list of similar items – [try the query for one item]
2. Given a user, the list of similar users– [try the query for one item]
3. The highest value of similarity (among users, among items)

Community detection

Community detection algorithms are used to evaluate how groups of nodes are clustered or partitioned, as well as their tendency to strengthen or break apart.

The Neo4j GDS library includes the several community detection algorithms.

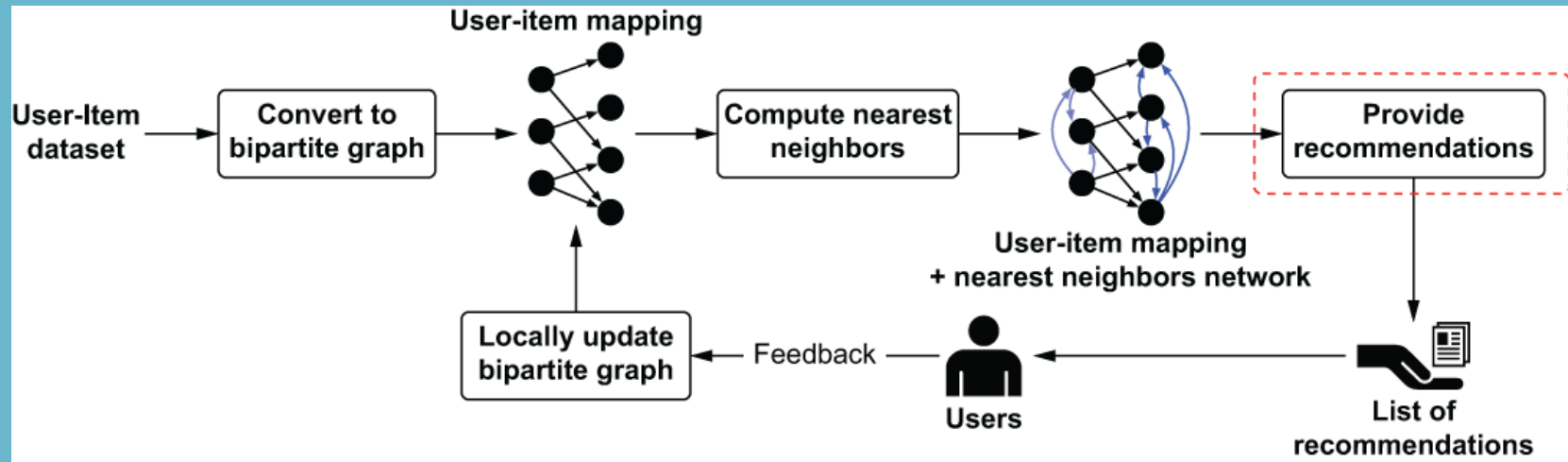
Some of the most robust and reliable, therefore suitable for use in real-world scenarios are:

- K-Means Clustering
- Label Propagation
- Local Clustering Coefficient
- Louvain
- Strongly Connected Components
- Triangle Count
- Weakly Connected Components

<https://neo4j.com/docs/graph-data-science/current/algorithms/community/>



**Providing
recommendat
ions**



Type of outputs

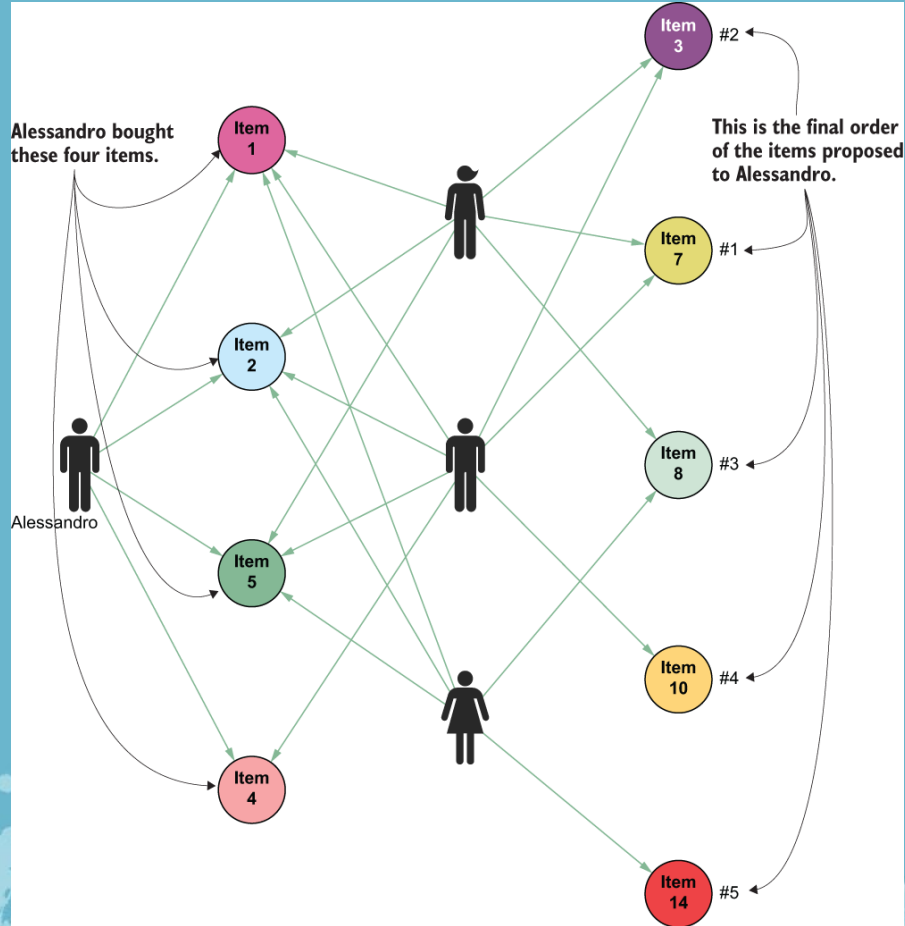
At a high level, the recommendation process produces the following types of output:

- **Relevance scores**—Numerical predictions indicating the degree to which the current user likes or dislikes a certain item.
- **Recommendations**—A list of N recommended items. Such a top-N list should not, of course, contain items that the current user has already bought unless the purchases can be recurring purchases.

User-based

- In the neighborhood approach to collaborative filtering, the first output—relevance scores—can be produced by looking at the user's nearest neighbors (the user-based approach) network or the item's nearest neighbors (the item-based approach) network.
- The basic idea in the user-based approach is that given the current user as input, the bipartite graph representing the interaction database, and the nearest neighbors network, for every product p that the user has not seen or bought, a prediction is computed based on the ratings for p made by the peer users (the users in that user's nearest neighbors network). Consider the following diagram

Collaborative filtering basic idea user-based approach



- The underlying assumptions of such methods [Jannach, et al., 2010] are
- If two users had similar tastes in the past, they will have similar tastes in the future.
 - Users' preferences remain stable and consistent over time.

Similarity

- Two cases can occur. In one case, the interactions between users and items (clicks, purchases, and views) contain no weight. This case is called the binary or Boolean model.
- In the second case, there is a weight to the interactions (such as ratings).

$$\text{pred}(a,p) = \frac{\sum_{b \in \text{KNN}(a)} \text{sim}(a,b)}{\sum_{b \in \text{KNN}(a)} 1} \times r_{b,p}$$

- This formula predicts the ratings that the user a would assign to the item p . $\text{KNN}(a)$ represents the k -nearest neighbors of the user a , and $r_{b,p}$ represents the rating assigned by the user b to the product p . This rating can be 0 if user b doesn't rate item p .

Similarity – Boolean case

The Boolean case is a little trickier, because in the literature, no single method is recognized as the best approach

$$score(a,p) = \frac{1}{|KNN(a)|} \times \sum_{b \in KNN(a)} r_{b,p}$$

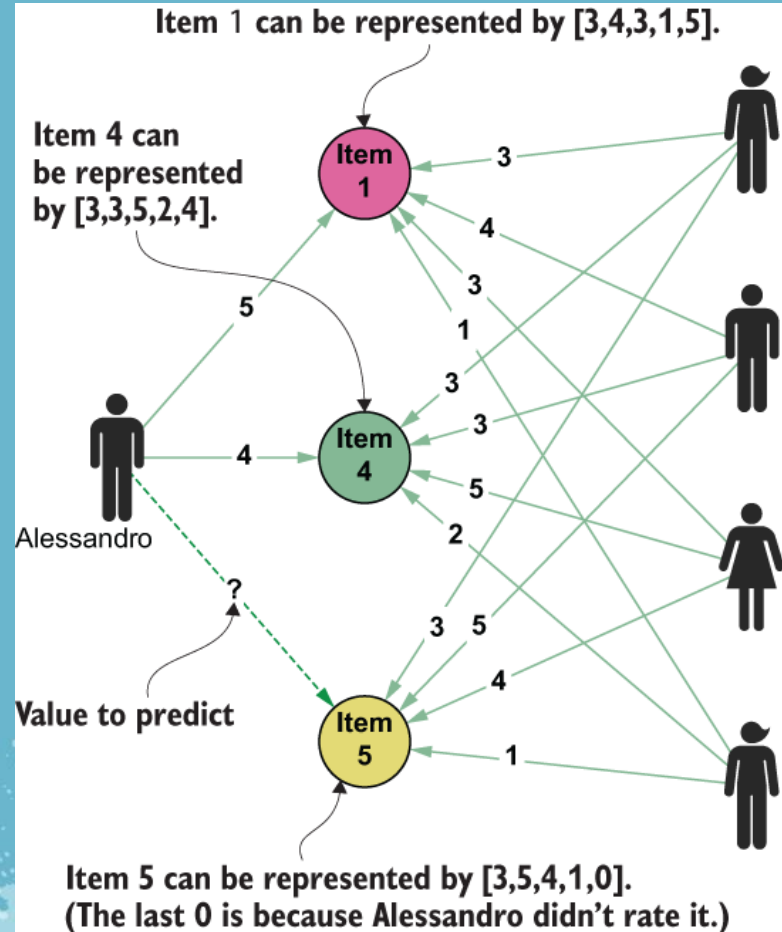
Here, $r_{b,p}$ will be 1 if the user b purchased the product p and 0 otherwise, so the sum will return how many of the nearest neighbors of a bought the product p . This value is normalized with the number of nearest neighbors for a (the value of $|KNN(a)|$).

The code is available in the `ch05/ recommendation/collaborative_filtering/ recommender.py` file.

WHAT ARE THE CHALLENGES?

Collaborative filtering –item based approach

- Item-based recommendations allow for the computation of recommendations in real time even for a large rating matrix [Sarwar et al., 2001].
- The main idea of item-based algorithms is to compute predictions by using the similarity between items, not users



Computing score

If the original User-Item dataset contains rating values, the formula to predict the rating for a not-yet-seen product in the dataset [Sarwar et al., 2001] is

$$\text{pred}(a, p) = \frac{\sum_{q \in \text{ratedItem}(a)} (\text{sim}(p, q) \times r_{a, q} \times |\text{KNN}(q) \cap \{p\}|)}{\sum_{q \in \text{ratedItem}(a)} (\text{sim}(p, q) \times |\text{KNN}(q) \cap \{p\}|)}$$

In our case,

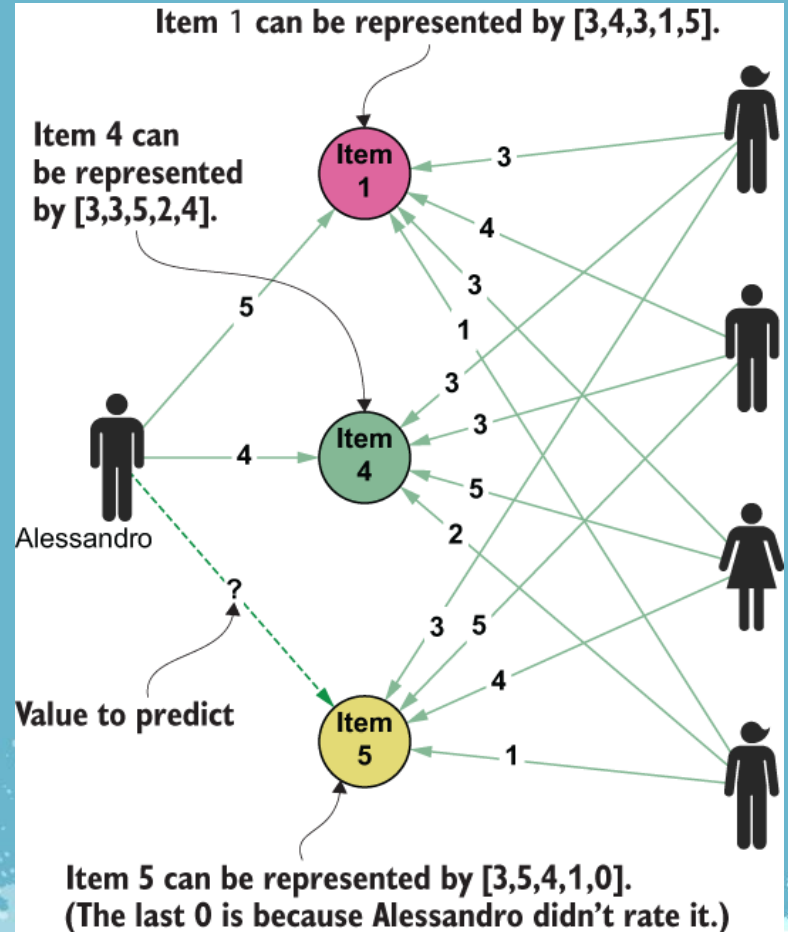
- $q \in \text{ratedItem}(a)$ considers all the products rated by user a .
- For each q , it multiplies three values:
 - o The similarity between q and the target product p .
 - o The rating assigned by the user to q .
 - o $|\text{KNN}(q) \cap \{p\}|$, which is 1 if p is in the set of nearest neighbors of q and 0 otherwise. It is possible to consider only the nearest neighbors of q and not all the similarities.
- The denominator normalizes the value to not exceed the max value of the rating.

Consider the data in figure, the similarities among all the three items are

- Item 1-Item 4: 0.943
- Item 1-Item 5: 0.759
- Item 4-Item 5: 0.811

us $|KNN(q) \cap \{p\}|$ is always 1. The user Alessandro rated Item 1 with 5 stars and Item 4 with 4 stars. The formula for predicting Alessandro's interest/stars for Item 5 is

$$pred(a,p) = \frac{0.759 \times 5 + 0.811 \times 4}{0.759 + 0.811} = \frac{7.039}{1.57} = 4.48$$



Computing score – boolean case

- As with the user-based approach, for the Boolean case there is no accepted standard approach for computing the scores, one option is [Deshpande and Karypis, 2004]:
-
- In this case, we have no ratings, so the formula is simpler. Furthermore, it doesn't return a prediction, but a generic score. Such a score can be normalized in different ways to facilitate comparisons.

The code is available in the `ch05/recommendation/ collaborative_filtering/recommender.py` file.

TODO @HOME

- Exercise
- Run the code in the preceding listing (which is available in the book's code repository), changing the users and observing how the list of recommendations changes. Then use a query to check the graph data base and see whether the suggested items are in line with previous items bought by a user.

- - - - - - In the Graph with User-based recommendation computed
- User-based recommendations for user 121688
- ['356475', '81345', '150318', '310791', '54058', '284871', '150100', '243566', '245400', '336832']
- User-based recommendations for user 114403
- ['356475', '15335', '81345', '150318', '310791', '54058', '284871', '150100', '243566', '245400']

The background is a solid teal color. It features several white, irregular splatters or paint-like marks scattered across the surface. There are also several thin, light blue diagonal lines that appear to be part of a larger graphic design, some in the top left and bottom right corners.

Dealing with the cold-start problem

Cold-start problem

- ! In real-world e-commerce applications, User-Item matrices tend to be sparse, as customers typically have bought (or have provided ratings for) only a small fraction of the products in the catalog.
- The problem is even worse for new users or new items that have had no or few interactions. This problem is referred to as the cold-start problem, and it further illustrates the importance of addressing the issue of data sparsity.
- *the challenge related to the cold-start problem is to compute good predictions when relatively little information is available*

Which approach can be applied to solve the cold start problem?

Possible Approaches

1 approach: to exploit additional information about the users, such as gender, age, education, interests, or any other available data that can help classify a user.

2 other approaches creates hybrid recommender systems that merge multiple approaches in a single prediction mechanism. These approaches are no longer purely collaborative, and new questions arise as to how to acquire the additional information and combine the different classifiers

Transitive associations method

A graph-based approach [Huang et al., 2004] explores transitive associations (similarities) among consumers through past transactions and feedback.

The main idea of this approach is to exploit the supposed transitivity of customer tastes when they share items. Users' preferences are represented by the items and their interactions with the items.

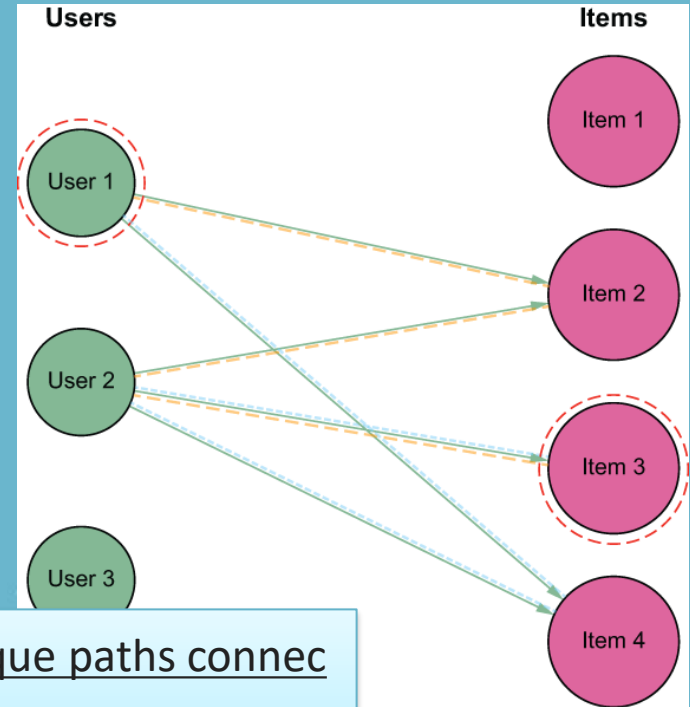
	Item 1	Item 2	Item 3	Item 4
User 1	0	1	0	1
User 2	0	1	1	1
User 3	1	0	1	0

Transitive associations method

In the transitive associations method, the recommendation approach can be easily implemented in a graph-based model by computing the associations between item nodes and user nodes.

The recommendations are determined by determining paths between users and items.

Standard collaborative filtering approaches, including both the user-based and item-based approaches, consider only paths with length equal to 3.

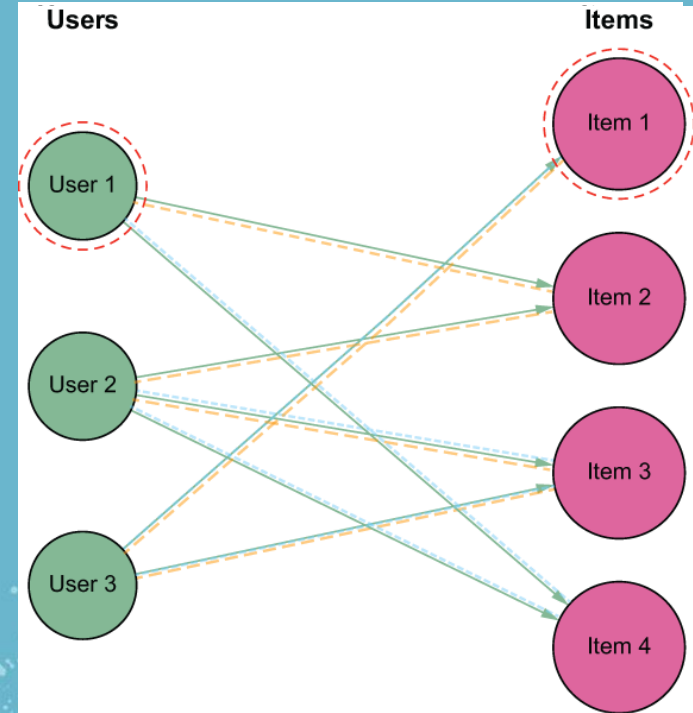


The higher the number of unique paths connecting an item node to a user node, the stronger the association is between those two nodes.

Transitive associations method

Considering indirect associations

Because the number of such paths of length 3 is small in sparse rating databases, the idea is to also consider longer paths—the so-called indirect associations—to compute recommendations. Extending the preceding approach to explore and incorporate transitive associations is straightforward in a graph-based model.



Scoring mechanism

It is necessary to define a scoring mechanism to order the recommendations list. In this case, recommendations are made based on the associations computed for pairs of user nodes and item nodes.

Given a user node User t and an item node Item j , the association between them, $\text{score}(\text{User } t, \text{Item } j)$, is defined as the sum of the weights of all distinctive paths connecting User t and Item j .

In this formula, only paths whose length is less than or equal to the maximum defined length M will be considered. The limit M is a parameter that the designer of the recommender system can control. (M has to be an odd number because transitive associations are represented in a bipartite graph)

$$\text{score}(\text{user}, \text{item}) = \sum_{p \in \text{pathsBetween}(\text{user}, \text{item}, M)} \alpha^{\text{length}(p)}$$

$\text{pathsBetween}(\text{user}, \text{item}, M)$ returns all the paths between user and item of length x where $x \leq M$.

advantages

This approach :

- generates high-quality recommendations even when a small amount of information is available.
- uses the same User-Item dataset used for the creation of the bi-graph.
- is purely graph-based, and it uses the same bi- graph model.

A comparison with the standard user-based and item-based algorithms shows that the quality of the recommendations can be significantly improved by the proposed technique based on indirect relationships, in particular when the ratings matrix is sparse.

SUMMARY (1/2)

The main aspects and advantages of the graph-based approach to collaborative filtering recommendation engines implemented with neighborhood methods are as follows:

- The User-Item dataset can easily be represented as a bipartite graph, in which the weight of each user-item pair is represented as an optional property of the relationship.
- The bipartite graph representation of the User-Item dataset not only has the advantage of allowing us to store only relevant information—avoiding wasting memory by storing useless zeros, as in the matrix representation—but also of speeding access during model creation by focusing only on the potentially relevant neighbors.
- It is possible to extract several vector representations for both items and users from the same graph model.

SUMMARY (2/2)

- The resulting model, consisting of similarities among users or items or both, can be naturally represented as new relationships that connect users and items. The resulting new graphs are the nearest neighbor (k-NN) networks.
- The algorithm, based on similarity computation, for creating the k-NN networks represents one of the most powerful and most widely adopted techniques for graph construction. The resulting networks not only are easy to navigate during the recommendation process, but also contain new knowledge distilled from the existing User-Item dataset that can be used for analyzing the data from other perspectives, such as item clustering, customer segmentation, and so on.
- A widely adopted technique for solving the data sparsity issue and the cold-start problem is based on graph representation, navigation, and processing. Even in this scenario, it is possible to mix multiple recommendation algorithms in a single graph and combine the power of multiple approaches to providing recommendations.

References

- [Bell et al., 2007] Bell, Yehuda Koren, and Chris Volinsky. “Modeling Relationships at Multiple Scales to Improve Accuracy of Large Recommender Systems.” Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2007): 95-104.
- [Börner, 2010] Börner, Katy. Atlas of Science: Visualizing What We Know. Cambridge, MA: MIT Press, 2010.
- [da Costa and Manzato, 2014] da Costa, Arthur F., and Marcelo Garcia Manzato. “Multimodal Interactions in Recommender Systems: An Ensembling Approach.” Proceedings of the Brazilian Conference on Intelligent Systems (2014): 67-72.
- [Deshpande and Karypis, 2004] Deshpande, Mukund, and George Karypis. “Item-Based Top-N Recommendation Algorithms.” ACM Transactions on Information Systems 22:1 (2004): 143-177. DOI: <http://dx.doi.org/10.1145/963770.963776>.
- [Diestel, 2008] Diestel, Reinhard. Graph Theory (Graduate Texts in Mathematics). 5th ed. Berlin: Springer, 2008.
- [Go et al., 2007] Go, Kwang-Il Goh, Michael E. Cusick, David Valle, Barton Childs, Marc Vidal, and Albert-László Barabási. “The Human Disease Network.” PNAS 104:21 (2007): 8685-8690. DOI: <https://doi.org/10.1073/pnas.0701361104>.
- [Goldberg et al., 1992] Goldberg, David, David Nichols, Brian M. Oki, and Douglas Terry. “Using Collaborative Filtering to Weave an Information Tapestry.” Communications of the ACM 35:12 (1992): 61-70. DOI: <http://doi.acm.org/10.1145/138859.138867>.
- [Grujić, 2008] Grujić, Jelena. “Movies Recommendation Networks as Bipartite Graphs.” Proceedings of the 8th International Conference on Computational Science, Part II (2008): 576-583.
- [Herlocker et al., 1999] Herlocker, Joseph A. Konstan, Al Borchers, and John Riedl. “An Algorithmic Framework for Performing Collaborative Filtering.” Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (1999): 230-237. DOI: <http://dx.doi.org/10.1145/312624.312682>.
- [Herlocker et al., 2004] Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. “Evaluating Collaborative Filtering Recommender Systems.” ACM Transactions on Information Systems 22:1 (2004): 5-53. DOI: <http://dx.doi.org/10.1145/963770.963772>.
- [Huang et al., 2004] Huang, Zan, Hsinchun Chen, and Daniel Zeng. “Applying Associative Retrieval Techniques to Alleviate the Sparsity Problem in Collaborative Filtering.” ACM Transactions on Information Systems 22:1 (2004): 116-142. DOI: <http://dx.doi.org/10.1145/963770.963775>.



Thank you